Deliverable 3: Conceptual Object Model (Entity Class Diagram) - Nestor

The conceptual object model describes the domain information required by ParkChoiceSG's driver, device-local and authenticated operator workflows. It is derived from UC-01 to UC-09 in the group's Lab 2 document, the relevant SRS sections (§4.1, §4.6 and §4.7), and the data dictionary. The model is presented as four views of one model for readability; a class repeated between views represents the same entity.

All classes carry the `«entity»` stereotype. The three named value vocabularies are shown as `«enumeration»` boxes. Associations show domain relationships and multiplicities. The filled diamond indicates composition: recommendation results belong to their search request. Attributes prefixed with `/` are derived. `[0..1]` indicates an optional attribute and `[0..*]` a collection. Domain entities may be transient; the stereotype does not require every class to become a database table. Boundary and control classes are documented separately in Deliverable 4.

3.1 Recommendations and parking data

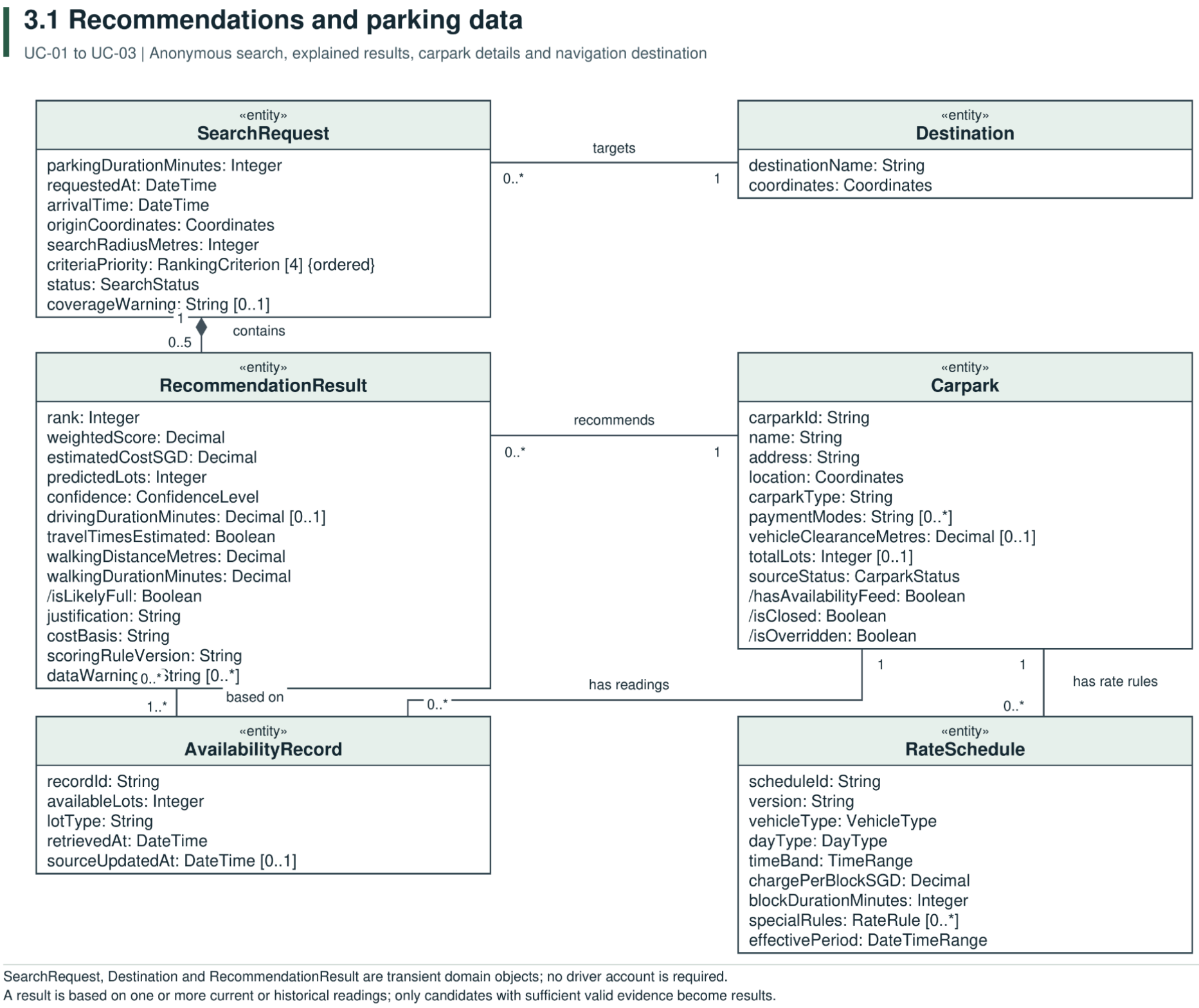


Figure 3.1. A `SearchRequest` captures `parkingDurationMinutes`, `requestedAt`, `arrivalTime`, `originCoordinates`, `searchRadiusMetres` and the driver's ordered `criteriaPriority`. It targets one `Destination` and contains zero to five `RecommendationResult` objects. Each result recommends one `Carpark`, stores its `weightedScore`, estimated cost, `drivingDurationMinutes` and walking measures, `predictedLots`, `confidence`, `travelTimesEstimated`, `/isLikelyFull` and an explanation, and is based on one or more `AvailabilityRecord` objects. A carpark may have multiple availability readings and applicable rate rules over time.

Parking duration and arrival time belong to the request because two searches for the same destination can use different values. `RecommendationResult` retains the evidence and calculation outputs for that request, including whether travel times were estimated and whether the carpark is likely full. A selected carpark is an existing `Carpark` in the active results, rather than a separate subclass. Its name, location, `carparkType`, `paymentModes` and derived `/hasAvailabilityFeed` support the detail view and the external-navigation handoff in UC-03.

The lower bound of zero on a carpark's readings and rate rules represents incomplete source data. It does not make an incomplete carpark eligible: a result requires sufficient valid availability, rate, route and prediction evidence for the selected criteria. A completed search has one to five results; an unsuccessful search has none. Results have distinct ranks from 1 to the number returned, with rank 1 having the highest weighted score under `criteriaPriority`.

3.2 Reports, authentication and overrides

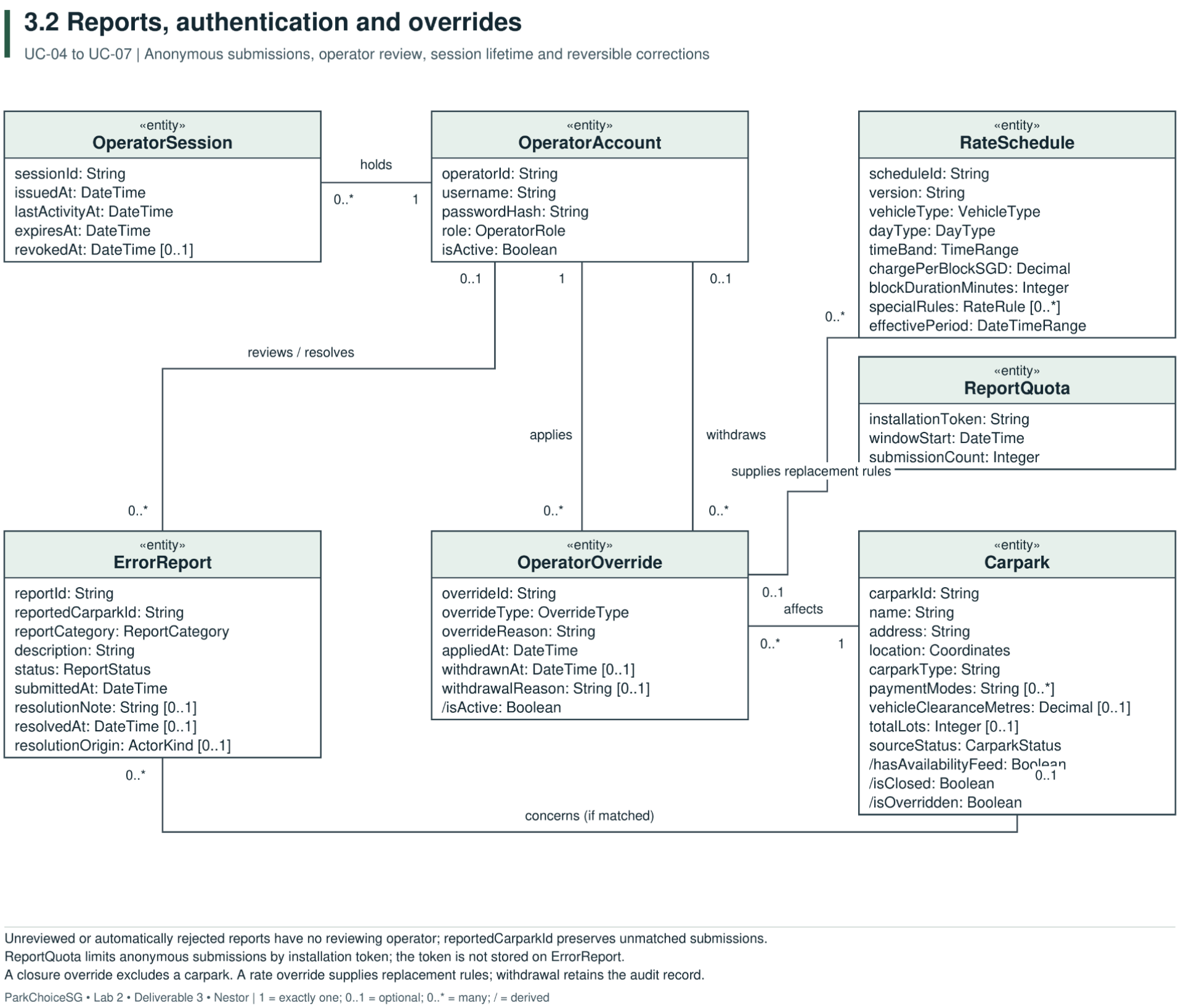


Figure 3.2. `ErrorReport` stores an anonymous submission and its eventual resolution. `ReportQuota` is a separate server-side quota record used to limit anonymous submissions within a time window; its installation token is not copied to `ErrorReport`, so the report remains unlinked to a device. `OperatorAccount` holds administrative identity and is associated with its `OperatorSession` records, reviewed reports, and applied or withdrawn `OperatorOverride` records. Each override affects exactly one carpark; a rate override supplies replacement `RateSchedule` rules.

A report can exist without a reviewing operator. The optional association also supports automatic rejection when the submitted carpark identifier cannot be matched, as described in UC-06.AC.1. `reportedCarparkId` preserves the originally submitted identifier even when no `Carpark` association exists. Operator resolutions record the reviewing account, mandatory remarks, resolution time and outcome. Automatically rejected reports record a system origin, reason and resolution time.

An override records its author, reason and application time. Withdrawal records the withdrawing operator, time and reason while preserving the original record. Closure and rate correction are distinct override types: a closure excludes the carpark, while a rate correction replaces its applicable charging rules. Source refreshes must preserve active overrides. `Carpark.isOverridden`, `Carpark.isClosed` and `OperatorOverride.isActive` are derived from the source status and override records rather than independently editable flags.

3.3 Source health and refresh records

3.3 Source health and refresh records

UC-08 and UC-09 | Per-feed freshness, operational measurements and manual or scheduled refresh attempts

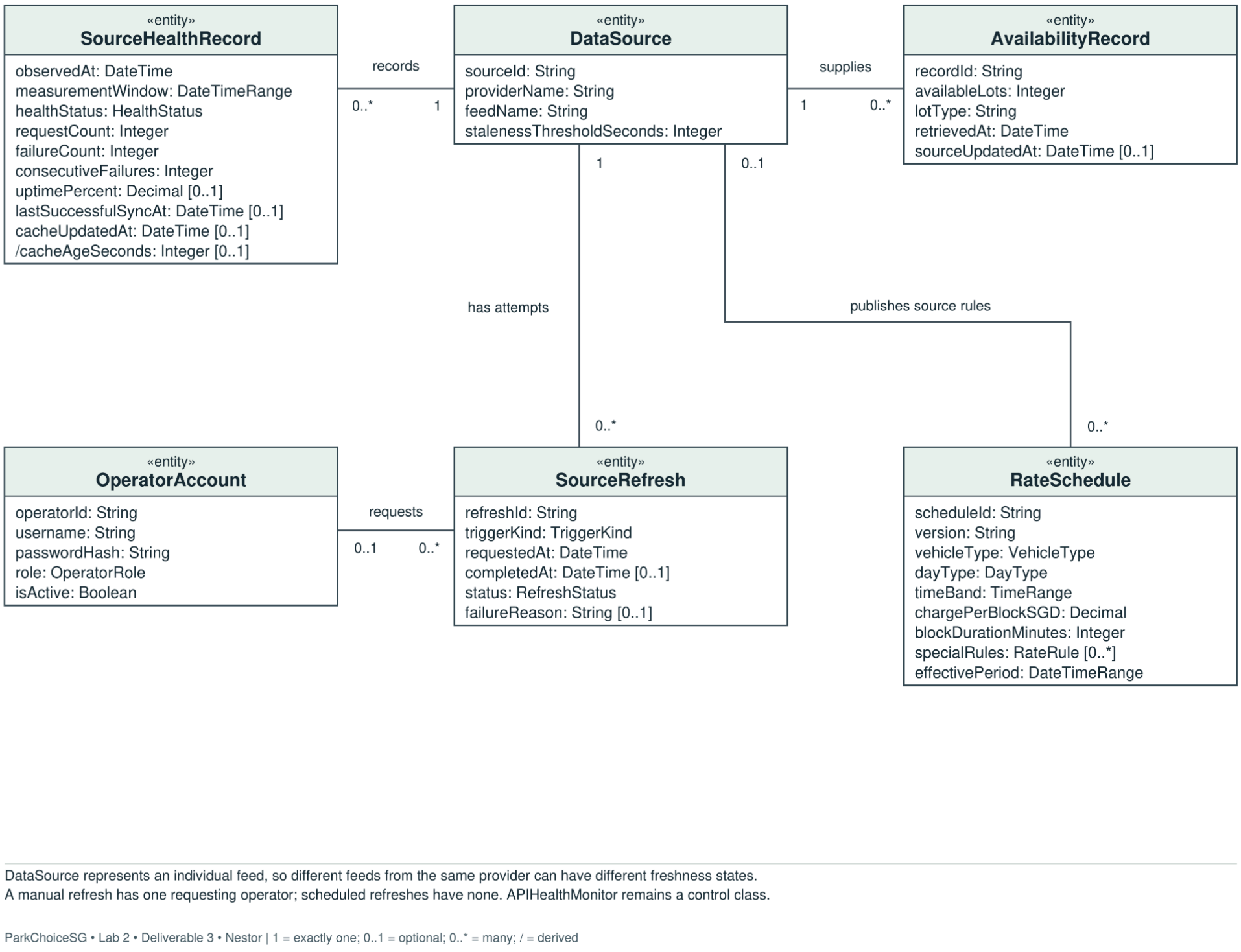


Figure 3.3. `DataSource` identifies an individual external feed. `SourceHealthRecord` contains the measured health and freshness information shown to operators. `SourceRefresh` records each refresh attempt and its outcome. Manual attempts reference the requesting operator; scheduled attempts do not.

Availability records identify their supplying source. Source-provided rate schedules identify their source; replacement rules from manual overrides are linked through `OperatorOverride` in Figure 3.2. A provider may expose multiple feeds with different refresh frequencies, so the model monitors feeds individually. `APIHealthMonitor` collects and interprets these records as a **control** class in Deliverable 4; it is not an entity.

3.4 Device-local data

3.4 Device-local data

Option A | Settings, saved places, search history, parking sessions, favourites and reminder support

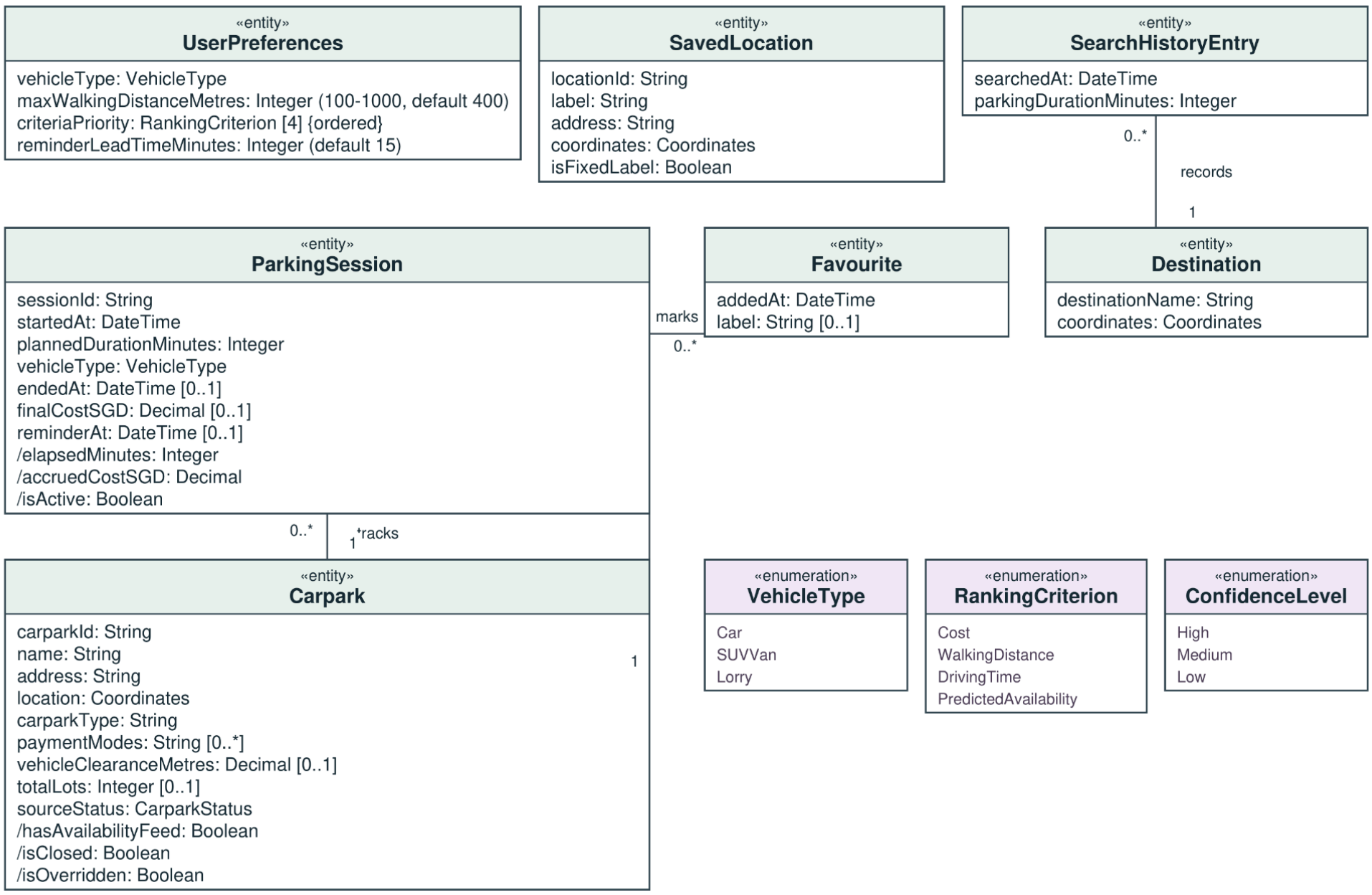


Figure 3.4. These five entities are stored only on the installation under device-local Option A (`FR-1.7`, `NFR-S5`); the backend does not hold them. `UserPreferences` has one instance per installation and supplies the vehicle type, walking limit, ordered ranking criteria and reminder lead time used by the relevant boundary/control classes. `SavedLocation` stores a labelled place without creating a server-side account. `SearchHistoryEntry` keeps at most ten recent searches for a destination. `ParkingSession` tracks a user's active or completed session at one carpark, while `Favourite` stores a saved carpark label. `VehicleType`, `RankingCriterion` and `ConfidenceLevel` are shared enumerations used by the entity attributes.

3.5 Entity responsibilities and use-case traceability

Entity	Responsibility	Use cases / requirements
SearchRequest	Captures duration, arrival context, search area, criteria priority and search outcome	UC-01; FR-2.5, FR-4.1, FR-4.11
Destination	Describes the resolved place being visited	UC-01, UC-02; FR-2.4
RecommendationResult	Retains ranked factors, weighted score, estimated cost, prediction confidence, explanation and evidence used	UC-01, UC-02; FR-4.5–FR-4.8, FR-4.12, FR-4.15
Carpark	Describes parking location, restrictions, payment options and effective operational status	UC-01–UC-04, UC-06, UC-07; FR-5.1, FR-5.4
AvailabilityRecord	Records a current or historical lot reading with source and retrieval time	UC-01, UC-02, UC-08, UC-09; FR-4.5, FR-4.15

RateSchedule	Describes versioned charging rules for a vehicle/day/time period	UC-01, UC-02, UC-07
ErrorReport	Records an anonymous issue and its resolution	UC-04, UC-06
ReportQuota	Counts anonymous submissions for an installation within a quota window	FR-7.7, BR-6
OperatorAccount	Identifies an authorised administrator by username	UC-05–UC-09
OperatorSession	Represents the lifetime and expiry of administrative access	UC-05; access precondition for UC-06–UC-09
OperatorOverride	Preserves a manual correction and its application/withdrawal audit information	UC-06, UC-07
DataSource	Identifies each external feed and its freshness threshold	UC-08, UC-09; evidence provenance in UC-01/UC-02
SourceHealthRecord	Stores a timed observation of source health and cache freshness	UC-08
SourceRefresh	Records a scheduled or manually requested refresh and outcome	UC-09, UC-08 alternative flow
UserPreferences	Stores the installation's vehicle, walking, ranking and reminder settings	FR-1.1–FR-1.4, FR-1.8, FR-6.4
SavedLocation	Stores a labelled device-local place	FR-1.5, FR-2.4
SearchHistoryEntry	Stores a recent destination search and duration	FR-2.6
ParkingSession	Tracks an active or completed device-local parking session	FR-6.1–FR-6.10
Favourite	Stores a device-local favourite carpark and optional label	FR-7.1, FR-7.2, FR-7.5

The model is derived from UC-01 to UC-09 together with SRS sections §4.1, §4.6 and §4.7. Where the revised SRS has not yet assigned a new use-case identifier, the traceability table uses the relevant FR/BR reference instead.

3.6 Model constraints and interpretation

1. Device-local scope. Driver sign-in and a backend driver profile are not required for these entities. UserPreferences, SavedLocation, SearchHistoryEntry, ParkingSession and Favourite are stored on the installation only. There is one preferences object per installation, history is capped at ten entries, and at most one parking session can be active.

2. Arrival and freshness. arrivalTime is the time used to select day/time rate rules and to evaluate travel and availability. AvailabilityRecord.retrievedAt is always recorded; sourceUpdatedAt is present only when the provider supplies it. The display must identify which timestamp it uses and label a fallback as retrieval time rather than source update time.

3. Prediction and ranking. criteriaPriority is an ordered list of the four RankingCriterion values. The weightedScore uses the selected priority across cost, walking distance, driving time and predicted availability. predictedLots and confidence preserve the prediction output; travelTimesEstimated identifies estimated routing values and isLikelyFull is derived. When history is insufficient, the current reading can be the only evidence used.

4. Rate rules. A RateSchedule represents a versioned day/time charging segment for a VehicleType. RateRule is an embedded domain value describing a verified grace period, free period or cap with its applicability and amount/duration. It is not an additional entity or a claimed complete billing schema. Time ranges can span midnight; effective periods distinguish rule versions. Unsupported or ambiguous charging rules make the estimate unavailable.

5. Override consistency. An active closure override has no replacement rate rules. An active rate override has one or more replacement rules, all belonging to the affected carpark. Each rate rule has exactly one provenance: an external DataSource or an OperatorOverride. Proposed constraint: at most one active override per carpark and override type; a later correction explicitly withdraws or supersedes the earlier one. On withdrawal, follow UC-07's next-refresh reconciliation rule and mark evidence unavailable if no trustworthy source value can yet be restored.

6. Quota privacy. ReportQuota.installationToken is a quota key held separately from ErrorReport; the report model does not store a device token or submitter identity. The configured quota window follows BR-6.

7. Report resolution. Proposed statuses are Pending, Resolved and Rejected. Pending reports need no reviewer or resolution fields. Closed reports require remarks, time and resolution origin; operator-origin resolutions require one operator. The model captures the final resolution, not a full multi-step moderation history.

8. Refresh outcome. Proposed refresh statuses are Requested, Running, Succeeded and Failed; trigger kind is Manual or Scheduled. Completion time exists only after completion. A failed refresh does not advance the last-successful-sync or cache-update time. A record is an analysis concept, not a commitment to a queue technology or indefinite retention of attempts.

9. Unknown values. Unknown capacity, height, uptime or cache age remains absent, never a fabricated zero. cacheAgeSeconds is derived from the observation time and cache update time, if a cache exists. Request/failure counts and uptime use the stated measurement window; consecutive failures represent the current run of failures.

10. Parking-tracker detail. zone and lotNumber are outside the current ParkingSession scope.

11. Value types and lifecycle. Coordinates contains latitude and longitude; TimeRange and DateTimeRange contain start/end values. The named status/category types are domain vocabularies to refine with the use cases. Account, carpark, report and override audit data require retention; device-local entities remain on the installation. Session, refresh and health-record retention is a later design decision. No inheritance or shared aggregation is added because the documented domain does not establish a suitable “is-a” hierarchy or shared whole/part relationship.